

2024 Sep Oppe 1 - Set 1

Section 2 - Problem 1

Sum of Squares of Even Numbers

Write a function `sum_of_squares_of_even` that takes a list of integers `nums` and returns the sum of squares of all even numbers in the list.

Example:

```
>>> nums = [1, 2, 3, 4, 5, 6]  
>>> sum_of_squares_of_even(nums)
```

56

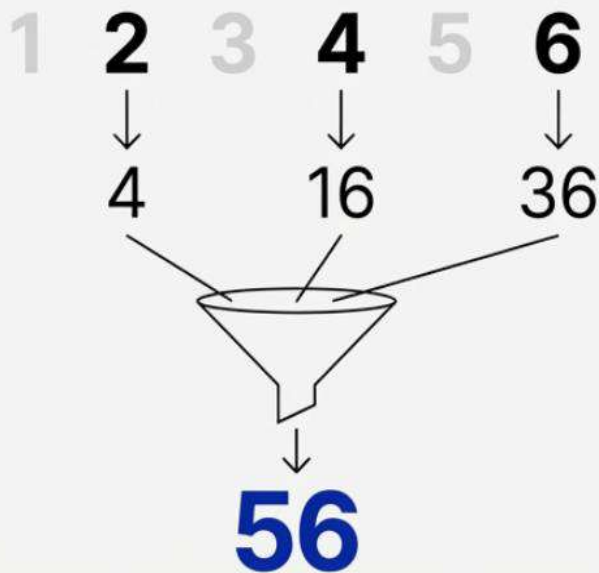
NOTE: This is a function type question, you don't have to take input or print the output, just have to complete the required function definition.

Explanation

In the list `[1, 2, 3, 4, 5, 6]`, the even numbers are 2, 4, and 6. The sum of their squares is $4 + 16 + 36 = 56$.

Algorithmic Summation of Squared Even Integers

Calculate the total sum after squaring only the even integers in a list.



Defining the Problem Scope

The Inputs

We accept a single list of integers, defined as variable 'nums'.

The Rules

1. **Identify:** Isolate all even numbers (Filter).
2. **Square:** Multiply each even number by itself (Transform).
3. **Sum:** Add the resulting squares together (Accumulate).

Example Data

Input: [1, 2, 3, 4, 5, 6]

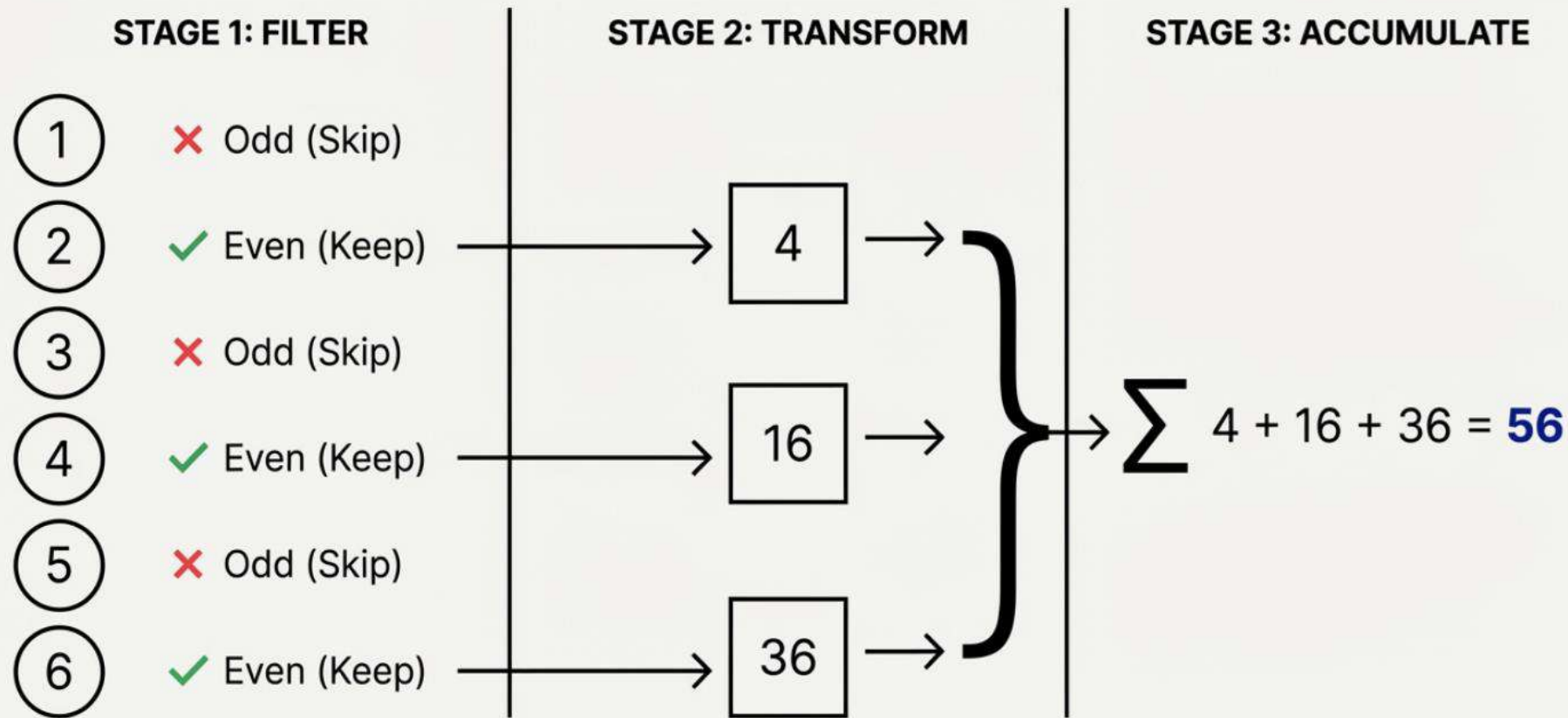
Filter Evens: 2, 4, 6

Square Calculation: $2^2 + 4^2 + 6^2$

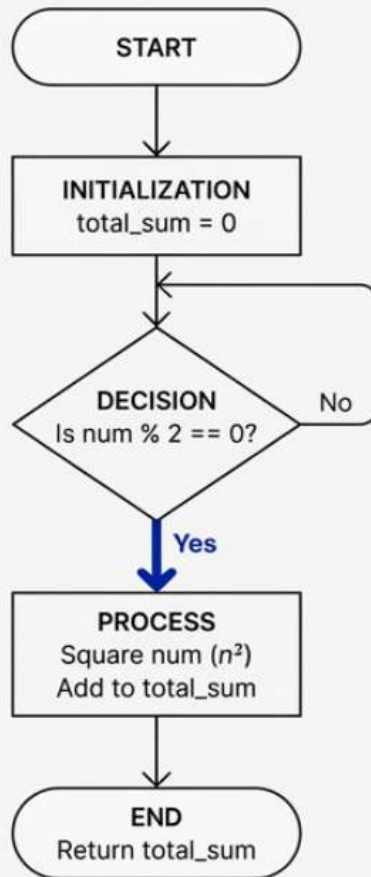
Intermediate: $4 + 16 + 36$

Result: 56

Visualizing the Data Pipeline



Logic Flowchart & Decision Path



The Explicit Solution: Manual Loop

```
def sum_of_squares_of_even(nums):
```

Initialization:
Starts with zero.

```
    total_sum = 0
```

Iteration: Loops
through every
item.

```
    for n in nums:
```

```
        if n % 2 == 0:
```

Filter: The Modulo
operator identifies
evens.

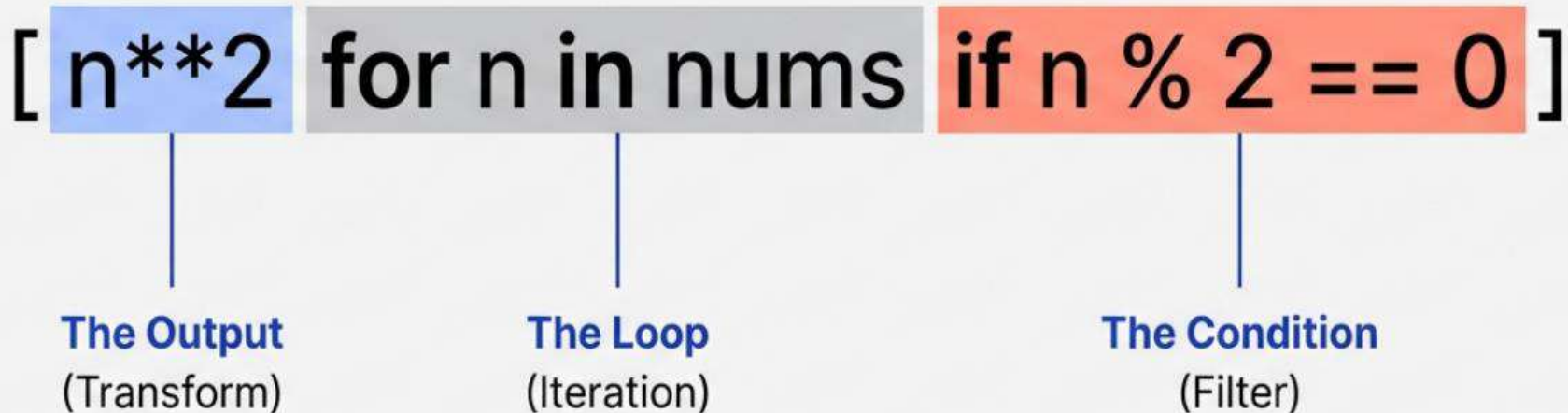
```
            total_sum += n**2
```

Accumulation:
Squares and adds
in one step.

```
    return total_sum
```

Moving From Logic to Elegance

Python allows us to combine the Filter and Transform steps into a single, readable line of code.



The Pythonic Solution

```
def sum_of_squares_of_even(nums: list) -> int:  
    # Most Pythonic solution using sum()  
    return sum(n**2 for n in nums if n % 2 == 0)
```

Key Note

The `sum()` function takes the iterable directly. This removes the need to manually initialize `total_sum = 0` or increment it line-by-line.

Optimization: List vs. Generator

List Comprehension

```
sum([n**2  
    for n in nums  
    if n % 2 == 0])
```

Memory

[4, 16, 36]

Constructs the entire list in memory before summing. Good for debugging.

Generator Expression

```
sum(n**2  
    for n in nums  
    if n % 2 == 0)
```

Memory

4 → 16 → 36

Lazily evaluates items one by one. No intermediate list is stored.

TAKEAWAY: Using a generator expression inside `sum()` avoids memory overhead. Best for large datasets.

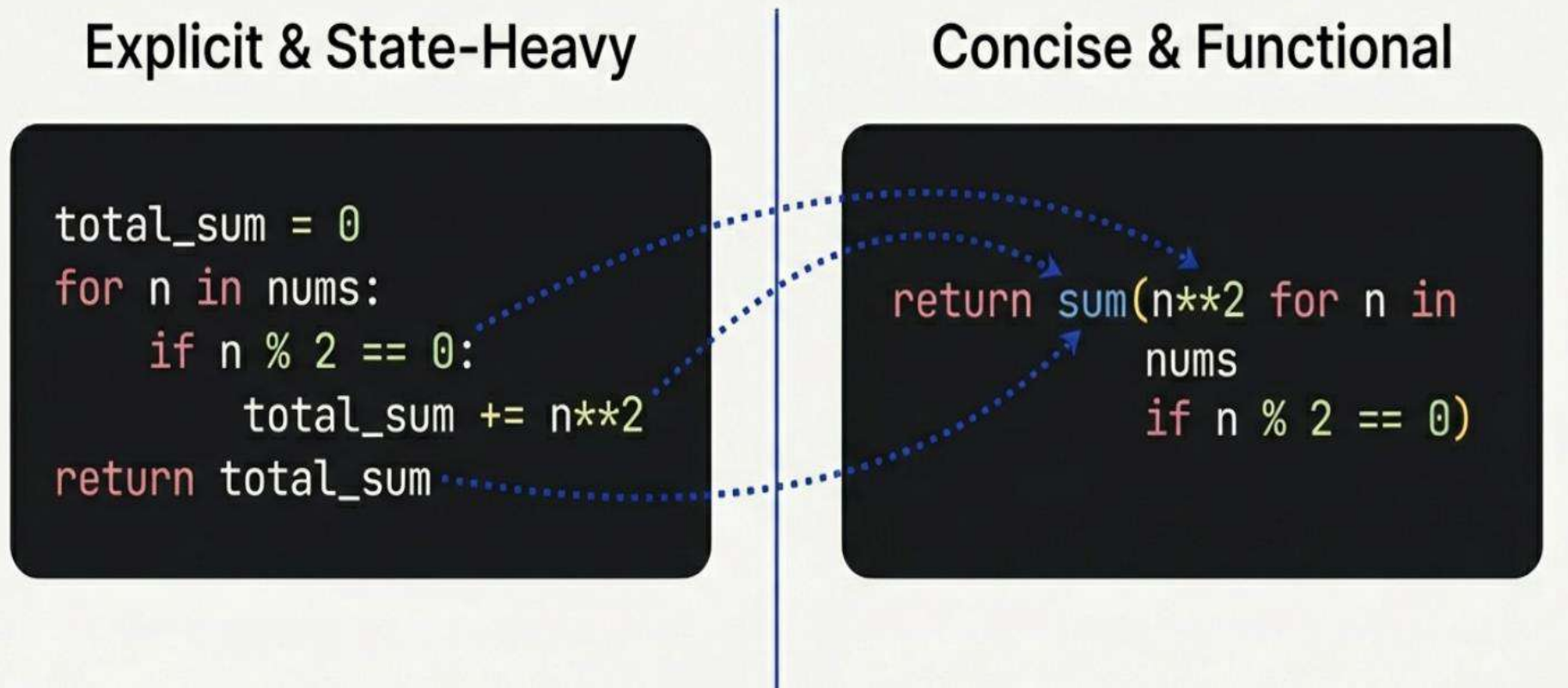
Code Evolution

Explicit & State-Heavy

```
total_sum = 0
for n in nums:
    if n % 2 == 0:
        total_sum += n**2
return total_sum
```

Concise & Functional

```
return sum(n**2 for n in
           nums
           if n % 2 == 0)
```

A vertical blue line separates the two code styles. Three blue dotted arrows point from the left code to the right code: one from 'total_sum = 0' to the opening parenthesis of the sum function, one from the 'if' condition to the filter part of the generator expression, and one from 'return total_sum' to the 'return' keyword.

Strategic Comparison Matrix

Approach	Pros	Primary Use Case
Manual Loop	Maximum Explicitness	Beginner learning & complex debugging
List Comprehension	Concise & Readable	Small lists where clarity is priority
Generator Expression	Memory Efficient	Production code & Big Data processing
Map/Filter	Functional Style	Strict functional programming environments

Engineering Best Practices



Filtering Rigor

Always use the modulo operator (`% 2 == 0`) to strictly identify even integers. Avoid loose typing.

```
if n % 2 == 0:
```



Memory Efficiency

Prioritize generator expressions for aggregation tasks. This prevents memory bloat when scaling to millions of inputs.

```
sum(n**2 for n in nums)
```



Readability First

Choose the approach your team can maintain. While one-liners are elegant, clarity should never be sacrificed for brevity.

```
return total_sum
```


Final Execution

```
$ python script.py  
>>> sum_of_squares_of_even([1, 2, 3, 4, 5, 6])  
>>> 56
```

From raw logic to efficient execution.